

Projektbericht

Lokale KI-Sprachassistent

Inhaltsverzeichnis

1. Einleitung	1
1.1. Hintergrund und Motivation	1
1.2. Zielsetzung des Projekts	1
2. Projektplanung	2
2.1. Projektorganisation	2
2.2. Kommunikation und Planung im Projekt	2
2.3. Funktionale Anforderungen	2
2.4. Nicht-funktionale Anforderungen	2
2.5. Projektplanung und Meilensteine	3
2.5.1. Projektphasen	3
2.5.2. Meilensteine	3
3. Technologien	4
3.1. Text To Speech (TTS)	4
3.2. Speech To Text (STT)	4
3.3. Authentifizierung	4
3.3.1. Zusammenfassung des Authentifizierungsmodells	5
3.4. Textklassifizierung	5
3.4.1. Methodik	5
3.4.2. Beispielhafte Umsetzung	5
3.4.3. Zusammenfassung des Textklassifizierungsmodells	6
3.5. Aktivierungswort	7
3.5.1. Zusammenfassung des Aktivierungswortmodells	7
3.6. PDF/Information Analyse	8
3.7. Verwendete Bibliotheken	8
3.7.1. Audioverarbeitung und Spracherkennung	8
3.7.2. Machine Learning und KI-Modelle	8
3.7.3. Web-Scraping und Web-Automatisierung	9
3.7.4. Datenverarbeitung und Visualisierung	9
3.7.5. Backend und API-Entwicklung	9
3.7.6. GUI-Entwicklung	9
3.7.7. Sonstige Hilfsbibliotheken	9
4. Systemarchitektur	10
4.1. Microservice-Architektur	10
4.2. Threading	10
4.3. Datenfluss und Verarbeitung	11
4.4. Datenbanken	11
4.4.1. Benutzerdatenbank	12
4.4.2. Aktivierungswortdatenbank	12

4.4.3. Textklassifizierungsdatenbank	13
5. Testing und Evaluation	15
5.1. Teststrategie und -methoden	15
5.2. Ergebnisse der Funktions- und Leistungstests	15
5.2.1. Ausgabeergebnisse	15
5.2.2. Konfusionsmatrix	17
6. Zukunftsperspektiven	24
6.1. Unterstützung für mehrere Benutzer	24
6.2. Verbesserte Spracherkennung	24
6.3. Bessere Informationsanalyse	24
6.4. Implementierung von LLMs	24
6.5. Erweiterung der Intents	24
7. Fazit	26
8. Anhang	27
8.1. Glossar	27
9. Literaturverzeichnis	28
9.1. Allgemeine Quellen zu KI	28
9.2. Audioverarbeitung und Feature-Extraktion	28
9.3. Microservices und Threading	28
9.4. Open-Source-Projekte	28
9.5. Frameworks und Tools	28
9.6. Datenschutz und Sicherheit	29
9.7. Hochschulbezogene Quellen	29
9.8. Sonstige Quellen	29

1. Einleitung

Die Entwicklung von Sprachassistenten hat in den letzten Jahren erhebliche Fortschritte gemacht und ist zu einem integralen Bestandteil moderner Technologien geworden. Von Smartphones über Smart-Home-Geräte bis hin zu autonomen Fahrzeugen – Sprachassistenten erleichtern die Interaktion zwischen Mensch und Maschine und bieten eine intuitive Bedienung. Lokale KI-Sprachassistenten bieten nicht nur eine schnellere Reaktionszeit, sondern auch verbesserten Datenschutz, da sensible Daten nicht an externe Server übertragen werden müssen.

1.1. Hintergrund und Motivation

Die Motivation für dieses Projekt liegt in der wachsenden Nachfrage nach datenschutzfreundlichen und effizienten Sprachassistenten. Cloud-basierte Lösungen, wie sie von großen Tech-Unternehmen angeboten werden, sind zwar leistungsstark, werfen jedoch Fragen hinsichtlich der Privatsphäre und der Abhängigkeit von Internetverbindungen auf. Ein lokaler KI-Sprachassistent, der auf dem Endgerät des Nutzers arbeitet, kann diese Probleme adressieren und gleichzeitig eine nahtlose Benutzererfahrung bieten.

1.2. Zielsetzung des Projekts

Das Ziel dieses Projekts ist die Entwicklung eines lokalen KI-Sprachassistenten, der in der Lage ist, Sprachbefehle zu verstehen, zu verarbeiten und auszuführen, ohne auf externe Cloud-Dienste angewiesen zu sein. Dabei sollen moderne Technologien der Künstlichen Intelligenz (KI) und des Natural Language Processing (NLP) eingesetzt werden, um eine effiziente und sichere Sprachverarbeitung zu gewährleisten. Der Assistent soll modular und erweiterbar sein, um zukünftige Anpassungen und Erweiterungen zu ermöglichen.

2. Projektplanung

2.1. Projektorganisation

Name	Studiengang	Rolle im Team
Erik Böhme	Wirtschaftsinformatik	Team Leiter, Suchintent Entwickler
Merte Laurentia Korpowski	Wirtschaftsinformatik	Studienordnungintent Entwickler
Tore Lemke	Wirtschaftsinformatik	Wikipediaintent Entwickler
Nathaniel Nathaniel	Wirtschaftsinformatik	KI-Modelle Entwickler
Nuha Alhamidi	Wirtschaftsinformatik	Todointent Entwickler
Anastasia Suglobow	Wirtschaftsinformatik	Todointent Entwickler

2.2. Kommunikation und Planung im Projekt

Die Kommunikation fand zu großen Teilen in den Meetings statt, wo auch das weitere Vorgehen und die nächsten Termine geplant wurden. Falls akut Probleme auftraten wurden diese in einer privaten WhatsApp-Gruppe besprochen und gelegentlich zusätzliche Meeting über Microsoft Teams abgehalten.

2.3. Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben, welche Funktionalitäten der lokale KI-Sprachassistent umfassen soll:

- **Spracherkennung:** Präzise Erkennung von Sprachbefehlen in Echtzeit.
- **Natürliche Sprachverarbeitung (NLP):** Verstehen und Verarbeiten von Benutzeranfragen.
- **Lokale Verarbeitung:** Alle Datenverarbeitung erfolgt auf dem Endgerät ohne Cloud-Anbindung.
- **Offline-Funktionalität:** Volle Funktionsfähigkeit ohne Internetverbindung.
- **Befehlsausführung:** Ausführen von Aufgaben wie Timer setzen, Erinnerungen erstellen, lokale Dateien suchen usw.
- **Datenschutz:** Keine Speicherung oder Übertragung von Benutzerdaten an Dritte.
- **Benutzerinteraktion:** Feedback an den Benutzer über Sprachausgabe oder visuelle Rückmeldungen.

2.4. Nicht-funktionale Anforderungen

Diese Anforderungen definieren, wie das System funktionieren soll:

- **Performance:** Echtzeitverarbeitung mit akzeptabler Latenz.
- **Benutzerfreundlichkeit:** Intuitive Bedienung und klare Sprachbefehle.
- **Robustheit:** Zuverlässige Funktionsweise auch bei Hintergrundgeräuschen.
- **Energieeffizienz:** Geringer Ressourcenverbrauch auf dem Endgerät.
- **Kompatibilität:** Unterstützung für verschiedene Betriebssysteme (z. B. Windows, Linux, macOS).

2.5. Projektplanung und Meilensteine

Die Projektplanung umfasst die zeitliche und organisatorische Strukturierung des Projekts:

2.5.1. Projektphasen

- **Analysephase:** Anforderungserhebung und Marktrecherche (1 Wochen).
- **Entwurfsphase:** Systemarchitektur und Technologieauswahl (1 Wochen).
- **Implementierungsphase & Testphase:** Entwicklung der Kernfunktionen (8 Wochen).
- **Evaluierungsphase:** Feedback-Einbindung (1 Wochen).
- **Abschlussphase:** Dokumentation und Präsentation (2 Wochen).

2.5.2. Meilensteine

- Meilenstein 1: !!! TODO !!!
- Meilenstein 2: !!! TODO !!!

Die erreichten Meilensteine wurden im Projekt durch eine agile Arbeitsweise schrittweise erarbeitet. Im Gegensatz zum Wasserfallmodell, bei dem alle Meilensteine von Anfang an festgelegt werden, haben sich diese im Laufe des Projekts dynamisch entwickelt. Die Meilensteine leiteten sich aus den definierten Zielen ab, die in regelmäßigen Meetings durch SOLL-IST-Wert-Überprüfungen überprüft und angepasst wurden.

3. Technologien

In diesem Abschnitt werden die wichtigsten Technologien und Komponenten beschrieben, die für die Entwicklung des lokalen KI-Sprachassistenten verwendet wurden. Dazu gehören Text-to-Speech, Speech-to-Text, Authentifizierung, Textklassifizierung, Aktivierungswort sowie die Implementierung einer PDF-/Informationsanalyse.

3.1. Text To Speech (TTS)

Für die Sprachausgabe (Text-to-Speech) wurde das Open-Source-Tool Coqui TTS verwendet. Als Datengrundlage diente das Thorsten Voice Dataset, das eine natürliche und hochwertige Sprachsynthese ermöglicht. Dieses Modell wurde gewählt, da es eine flexible Anpassung an die spezifischen Anforderungen des Projekts bietet und gleichzeitig eine hohe Qualität der Sprachausgabe gewährleistet.

3.2. Speech To Text (STT)

Die Spracherkennung (Speech-to-Text) wurde mit dem Open-Source-Modell Whisper umgesetzt. Whisper zeichnet sich durch seine hohe Genauigkeit und Robustheit bei der Umwandlung von gesprochener Sprache in Text aus. Es unterstützt mehrere Sprachen und eignet sich besonders gut für lokale Anwendungen, da es ohne Cloud-Anbindung betrieben werden kann.

3.3. Authentifizierung

Um eine sichere und benutzerfreundliche Authentifizierung zu ermöglichen, wurde ein 1D-Convolutional Neural Network (CNN)-Modell implementiert. Dieses Modell dient dazu, Nutzer anhand ihrer Stimme zu identifizieren und zu authentifizieren. Mit 11.561 Parametern ist das Modell effizient und dennoch leistungsstark genug, um eine hohe Genauigkeit bei der Benutzererkennung zu gewährleisten.

Als Datengrundlage wurden verschiedene akustische Merkmale extrahiert, darunter:

- Sprachstruktur (MFCC)
- Tonalität (Chroma)
- Klangenergie (Mel-Spektrogramm)
- Spektrale Unterschiede (Kontrast)
- Harmonie (Tonnetz)

Diese Merkmale wurden zu einem einzigen Array mit einer Länge von 193 Werten kombiniert, das als Eingabe für das Modell dient. Dieser Ansatz ermöglicht eine robuste und zuverlässige Spracherkennung sowie eine präzise Benutzeridentifikation, selbst in anspruchsvollen akustischen Umgebungen.

3.3.1. Zusammenfassung des Authentifizierungsmodells

Layer (type:depth-idx)	Output Shape	Param #
Model	[10000, 1]	--
Conv1d: 1-1	[10000, 20, 193]	80
Conv1d: 1-2	[10000, 10, 193]	610
MaxPool1d: 1-3	[10000, 10, 96]	--
Conv1d: 1-4	[10000, 20, 96]	620
Conv1d: 1-5	[10000, 10, 96]	610
MaxPool1d: 1-6	[10000, 10, 48]	--
Linear: 1-7	[10000, 20]	9,620
Dropout: 1-8	[10000, 20]	--
Linear: 1-9	[10000, 1]	21
Total params: 11,561		
Trainable params: 11,561		
Non-trainable params: 0		
Total mult-adds (Units.GIGABYTES): 2.61		
Input size (MB): 7.72		
Forward/backward pass size (MB): 695.28		
Params size (MB): 0.05		
Estimated Total Size (MB): 703.05		

Abbildung 1. Zusammenfassung des Authentifizierungsmodells

3.4. Textklassifizierung

In diesem Projekt haben wir ein BERT-Modell mit einem eigenen Datensatz feinabgestimmt, um die Textklassifizierung zu verbessern. Dabei setzen wir das IOB2-Format zur Annotation ein und implementierten eine feinere Unterteilung der Intents für eine präzisere Klassifikation.

3.4.1. Methodik

Unser Forschungsansatz basiert auf einer zweigeteilten Intent-Klassifikation:

1. **Anmerkung (Entität):** Zur Markierung spezifischer Elemente innerhalb eines Satzes.
2. **Intent:** Aufgeteilt in zwei Kategorien:
 - **Szenario:** Das übergeordnete Thema der Anfrage
 - **Absicht:** Das zugehörige Verb bzw. die Aktion

Diese Struktur ermöglicht eine verbesserte Qualität der KI, da die Unterteilung eine präzisere Erkennung von Objekten und Verben erlaubt.

3.4.2. Beispielhafte Umsetzung

Zur Veranschaulichung der Funktionsweise wird ein Trainingsbeispiel herangezogen, bei dem die KI ausschließlich auf die Erkennung von Datumsanfragen trainiert wurde

Fall 1: Getrennte Intents

Eingabe	Szenario	Absicht
Welches Datum ist jetzt?	Datum	Abfragen
Welche Uhrzeit ist jetzt?	(Nicht erkannt / unbekannt)	Abfragen

- In diesem Fall wird die Absicht „Abfragen“ korrekt erkannt, obwohl das Szenario („Uhrzeit“) nicht explizit trainiert wurde. Die KI versteht die Anfrage dennoch, da die übergeordnete Absicht („Abfragen“) zutrifft.

Fall 2: Ungetrennte Intents

Eingabe	Intent
Welches Datum ist jetzt?	Datum Abfragen
Welche Uhrzeit ist jetzt?	(Nicht erkannt / unbekannt)

- Hier zeigt sich eine Schwäche der starren Intent-Struktur: Da die KI nicht auf die Erkennung von „Uhrzeit“ trainiert wurde, kann sie die Anfrage nicht korrekt interpretieren. Dies verdeutlicht, wie wichtig eine differenzierte und umfassende Trainingsdatenbasis ist, um die Robustheit des Systems zu gewährleisten.

Durch die Aufteilung des Intents in "Szenario" und "Absicht" konnten wir die Flexibilität und Genauigkeit der KI verbessern. Diese Struktur ermöglicht eine robustere Generalisierung und eine präzisere Klassifikation, insbesondere bei unbekannten oder leicht abweichenden Anfragen.

3.4.3. Zusammenfassung des Textklassifizierungsmodells

Layer (type:depth-idx)	Output Shape	Param #
Model	[50, 70, 11]	--
└BertModel: 1-1	[50, 768]	--
└BertEmbeddings: 2-1	[50, 70, 768]	--
└Embedding: 3-1	[50, 70, 768]	23,886,336
└Embedding: 3-2	[50, 70, 768]	1,536
└Embedding: 3-3	[1, 70, 768]	393,216
└LayerNorm: 3-4	[50, 70, 768]	1,536
└Dropout: 3-5	[50, 70, 768]	--
└BertEncoder: 2-2	[50, 70, 768]	--
└ModuleList: 3-6	--	85,054,464
└BertPooler: 2-3	[50, 768]	--
└Linear: 3-7	[50, 768]	590,592
└Tanh: 3-8	[50, 768]	--
└Dropout: 1-2	[50, 70, 768]	--
└Dropout: 1-3	[50, 768]	--
└Dropout: 1-4	[50, 768]	--
└Linear: 1-5	[50, 70, 11]	8,459
└Linear: 1-6	[50, 7]	5,383
└Linear: 1-7	[50, 9]	6,921
Total params: 109,948,443		
Trainable params: 109,948,443		
Non-trainable params: 0		
Total mult-adds (Units.GIGABYTES): 5.48		
Input size (MB): 0.08		
Forward/backward pass size (MB): 2904.09		
Params size (MB): 439.79		
Estimated Total Size (MB): 3343.97		

Abbildung 2. Zusammenfassung des Textklassifizierungsmodells

3.5. Aktivierungswort

Zur Erkennung des Aktivierungsworts wurde ein 2D-Convolutional Neural Network (CNN)-Modell implementiert. Dieses Modell wurde speziell für die Verarbeitung von Mel-Spektrogrammen trainiert, die aus den Audiodaten generiert wurden. Mit insgesamt 7.499.457 Parametern bietet das Modell eine hohe Genauigkeit bei der Identifikation des Aktivierungsworts, selbst in Umgebungen mit Hintergrundgeräuschen. Die Verwendung von Mel-Spektrogrammen als Eingabedaten ermöglicht eine effiziente und robuste Merkmalsextraktion, was die Erkennungsleistung deutlich verbessert.

3.5.1. Zusammenfassung des Aktivierungswortmodells

Layer (type:depth-idx)	Output Shape	Param #
Model	[50, 1]	--
Conv2d: 1-1	[50, 32, 128, 63]	320
BatchNorm2d: 1-2	[50, 32, 128, 63]	64
MaxPool2d: 1-3	[50, 32, 64, 31]	--
Conv2d: 1-4	[50, 64, 64, 31]	18,496
BatchNorm2d: 1-5	[50, 64, 64, 31]	128
MaxPool2d: 1-6	[50, 64, 32, 15]	--
Conv2d: 1-7	[50, 128, 32, 15]	73,856
BatchNorm2d: 1-8	[50, 128, 32, 15]	256
MaxPool2d: 1-9	[50, 128, 16, 7]	--
Linear: 1-10	[50, 512]	7,340,544
Dropout: 1-11	[50, 512]	--
Linear: 1-12	[50, 128]	65,664
Dropout: 1-13	[50, 128]	--
Linear: 1-14	[50, 1]	129
Total params: 7,499,457		
Trainable params: 7,499,457		
Non-trainable params: 0		
Total mult-adds (Units.GIGABYTES): 4.11		
Input size (MB): 1.61		
Forward/backward pass size (MB): 357.43		
Params size (MB): 30.00		
Estimated Total Size (MB): 389.04		

Abbildung 3. Zusammenfassung des Aktivierungswortmodells

3.6. PDF/Information Analyse

Für die Analyse von PDF-Dokumenten und anderen textbasierten Informationen wurde das Open-Source-Modell Llama 3.2 integriert. Dieses Modell ermöglicht die Extraktion und Interpretation von relevanten Informationen, wie z. B. Studienordnungen oder Wikipedia-Inhalten. Es wurde speziell für die Erkennung von Intents wie StudienordnungIntent und WikipediaIntent trainiert, um gezielte und präzise Antworten auf Benutzeranfragen zu liefern.

3.7. Verwendete Bibliotheken

Für die Implementierung unseres Projekts wurden folgende Bibliotheken genutzt:

3.7.1. Audioverarbeitung und Spracherkennung

- **librosa==0.10.2.post1**: Analyse und Verarbeitung von Audiodaten (z. B. Feature-Extraction für Sprachdaten).
- **PyAudio==0.2.14**: Aufnahme und Wiedergabe von Audiodaten in Echtzeit.
- **openai_whisper==20240930**: Modell für Spracherkennung (Speech-to-Text).

3.7.2. Machine Learning und KI-Modelle

- **scikit_learn==1.5.2**: Machine-Learning-Algorithmen für Klassifikation, Regression und Datenverarbeitung.

- **torchmetrics==1.6.0:** Metriken zur Bewertung von Modellen (z. B. Genauigkeit, F1-Score).
- **transformers==4.46.1:** Nutzung von State-of-the-Art NLP-Modellen (z. B. BERT).
- **torchinfo==1.8.0:** Visualisierung und Analyse von PyTorch-Modellen.
- **mlxtend==0.23.3:** Hilfsfunktionen für Machine Learning und Datenanalyse.
- **nlTK==3.9.1:** Verarbeitung und Analyse von Textdaten (Tokenisierung, Stemming, etc.).
- **langchain-ollama==0.2.0:** Framework für die Erstellung von KI-gestützten Sprachassistenten und NLP-Pipelines.

3.7.3. Web-Scraping und Web-Automatisierung

- **selenium==4.26.1:** Automatisierung von Webbrowsern für das Extrahieren von Daten oder Interaktion mit Webseiten.
- **playwright==1.48.0:** Modernes Tool zur Browser-Automatisierung und Testing.
- **webdriver_manager==4.0.2:** Verwaltung von WebDriver-Instanzen für Selenium.
- **beautifulsoup4==4.12.3:** Parsen von HTML- und XML-Daten für Web-Scraping.
- **Requests==2.32.3:** HTTP-Anfragen für das Abrufen von Webinhalten.

3.7.4. Datenverarbeitung und Visualisierung

- **matplotlib==3.9.2:** Erstellung von Diagrammen und Visualisierungen.
- **pdfplumber==0.11.4:** Extrahieren und Analysieren von Daten aus PDF-Dateien.
- **joblib==1.4.2:** Effiziente Speicherung und Laden von großen Datenmengen (z. B. Modelle).

3.7.5. Backend und API-Entwicklung

- **fastapi==0.115.6:** Framework zur Erstellung von APIs für den Sprachassistenten.
- **uvicorn==0.32.1:** ASGI-Server für die Bereitstellung der FastAPI-Anwendung.

3.7.6. GUI-Entwicklung

- **pywebview==5.2:** Erstellung von einfachen GUI-Anwendungen mit Webtechnologien.

3.7.7. Sonstige Hilfsbibliotheken

- **pytz==2024.1:** Zeitonenunterstützung für Datums- und Zeitoperationen.
- **wikipedia==1.4.0:** Zugriff auf Wikipedia-Daten für Informationsbeschaffung.

4. Systemarchitektur

4.1. Microservice-Architektur

Eine zentrale Anforderung an die Architektur bestand darin, einen lokalen Sprachassistenten mit einem leichtgewichtigen Client im Frontend zu entwickeln. Im Backend sollten zahlreiche Mikroservices implementiert werden. Zu den geplanten Mikroservices gehörten unter anderem:

- Main-Server
- Accountverwaltung
- Textklassifizierung
- Speech-to-Text
- Text-to-Speech
- Datum-Intent
- Uhrzeit-Intent
- Wetter-Intent
- Wikipedia-Intent
- Studienordnung durchsuchen-Intent
- ToDo-Intent
- Google-Search-Intent
- YouTube-Intent (zunächst ein DuckDuckGo-Suchintend)
- Todo-Intend

Ursprünglich war ein DuckDuckGo-Suchintend geplant, jedoch wurde dieser aufgrund eines Rate-Limits in der DuckDuckGo-API, das die Nutzung stark einschränkte, durch einen YouTube-Suchintend ersetzt. Das Rate-Limit erlaubte nur drei Suchanfragen in kurzer Zeit, was zu erheblichen Einschränkungen und Problemen beim Testen führte. Der Wechsel zu YouTube ermöglichte eine zuverlässigere und effizientere Umsetzung des Suchintends.

4.2. Threading

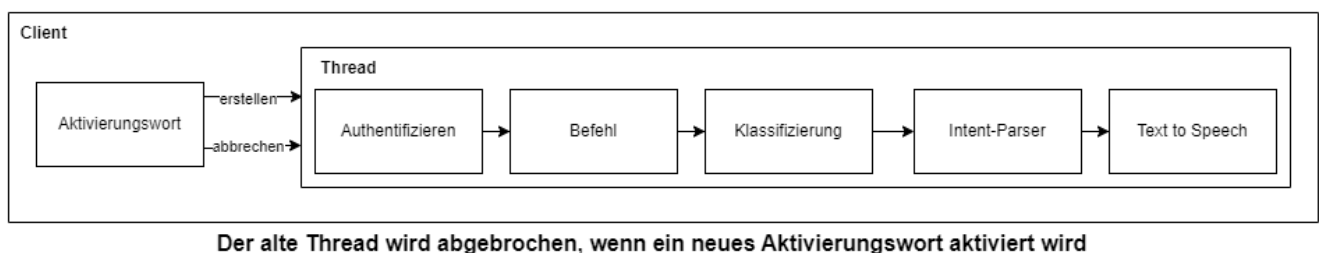


Abbildung 4. Threading

Das Multithreading in Python stellt kein echtes Multithreading dar, da es lediglich zwischen verschiedenen Prozessen wechselt. Dieser Wechsel erfolgt jedoch so schnell, dass er für den

normalen Nutzer nicht wahrnehmbar ist. In diesem Projekt wurde Multithreading eingesetzt, um eine benutzerfreundliche Erfahrung zu ermöglichen. Der lokale Sprachassistent kann eine laufende Ausgabe unterbrechen, sobald ein Aktivierungswort erkannt wird, und sofort die neue Anfrage bearbeiten, sodass der Nutzer nicht auf das Ende des vorherigen Befehls warten muss, um einen neuen Befehl einzugeben.

4.3. Datenfluss und Verarbeitung

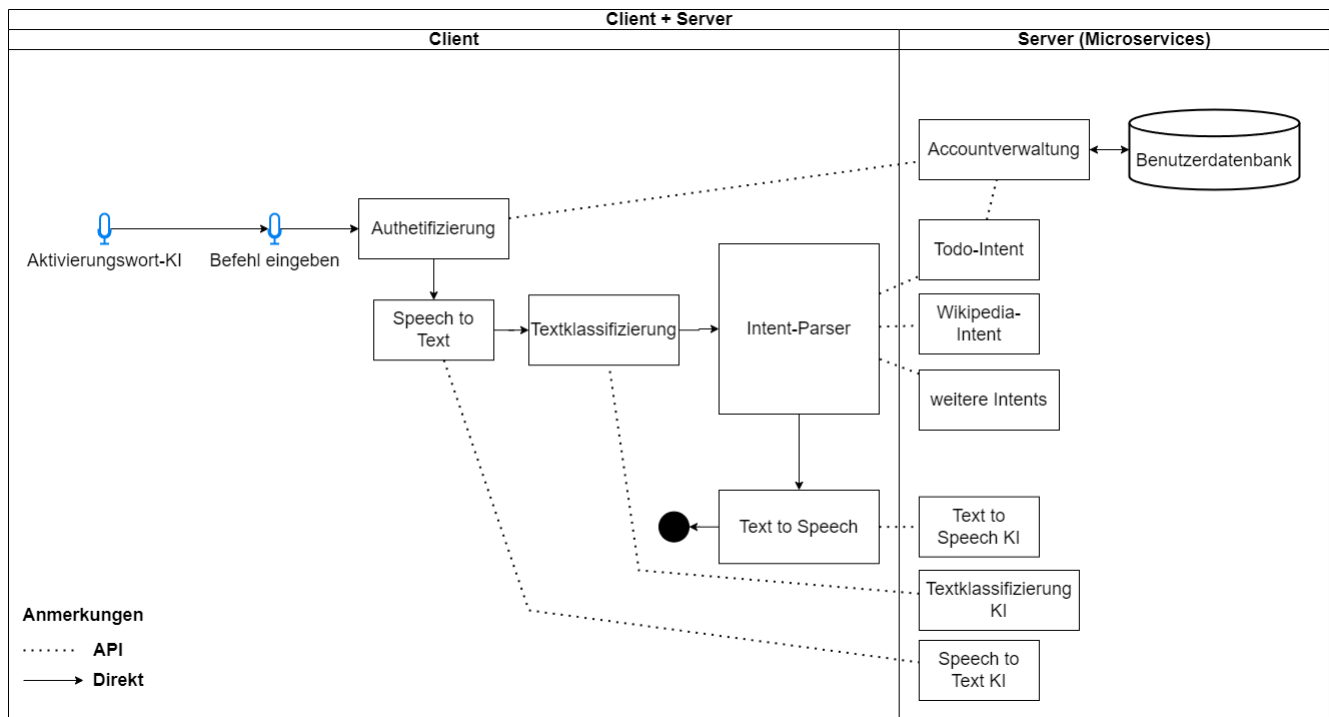


Abbildung 5. Aktivitätsdiagramm

Das Aktivierungswort dient dazu, den Benutzer zu erkennen. Dieser Prozess findet clientseitig statt. Zunächst wird der Client authentifiziert und sendet eine Sprachaufnahme an den Mikroservice der Accountverwaltung. Dieser prüft mithilfe der Authentifizierungs-KI, ob der Sprecher erkannt wurde, und antwortet mit der Benutzer-ID, falls die Erkennung erfolgreich war.

Nach der Sprechererkennung wird der Sprachbefehl (die Anfrage an den Sprachassistenten) mithilfe von Whisper von Speech-to-Text in Text umgewandelt. Anschließend klassifiziert BERT den Text, um den passenden Intent zu ermitteln. Der Intent-Parser ruft den entsprechenden Mikroservice über einen API-Request auf und empfängt die Antwort. Diese Antwort wird durch Text-to-Speech (TTS) in Sprache umgewandelt und als Sprachantwort an den Nutzer zurückgegeben.

4.4. Datenbanken

Im Rahmen des Projekts zur Entwicklung eines lokalen KI-Sprachassistenten wurden drei Datenbanken angelegt, um die erforderlichen Daten effizient zu speichern und zu verwalten. Jede Datenbank erfüllt spezifische Anforderungen und unterstützt die Funktionalitäten des Systems. Für die Implementierung wurde SQLite3 als Datenbankmanagementsystem gewählt, da es leichtgewichtig, einfach zu integrieren und ideal für lokale Anwendungen geeignet ist. Im Folgenden wird die Struktur und der Zweck jeder Datenbank beschrieben.

4.4.1. Benutzerdatenbank

Diese Datenbank dient der Speicherung von Informationen, die für die Authentifizierung und Personalisierung der Nutzer erforderlich sind. Sie enthält Daten wie die Merkmale der Stimme des Nutzers, die für die Authentifizierung durch die KI verwendet werden. Zusätzlich werden hier die persönlichen Daten der Nutzer, wie z.B. der Name, sowie ihre individuellen To-Do-Listen gespeichert. Diese Datenbank ist zentral für die Benutzerverwaltung und die Bereitstellung personalisierter Dienste.

Schema

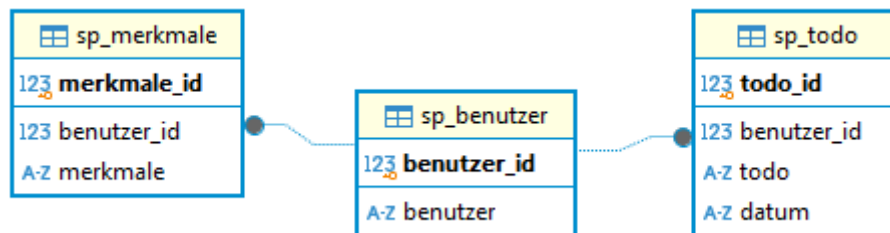


Abbildung 6. Benutzerdatenbank

Inhaltsbeispiel

A-Z benutzer	A-Z todo	A-Z merkmale
Nathaniel	Joggen	[758.1207275390625, 63.475589752197266, 7.5508341789245605, 48.335174560546875, -20.040817260742188, 7.150524139
Nathaniel	Meeting	[758.1207275390625, 63.475589752197266, 7.5508341789245605, 48.335174560546875, -20.040817260742188, 7.150524139

Abbildung 7. Benutzerdatenbankinhalt

4.4.2. Aktivierungswortdatenbank

In dieser Datenbank werden die Merkmale der Stimme gespeichert, die für die Erkennung des Aktivierungsworts durch die KI erforderlich sind. Das Aktivierungswort ist ein Schlüsselement des Systems, da es den Sprachassistenten aktiviert und die Interaktion einleitet. Die gespeicherten Daten dienen als Trainingsgrundlage für die KI, um das Aktivierungswort zuverlässig und sicher zu erkennen.

Schema



Abbildung 8. Aktivierungswortdatenbank

Inhaltsbeispiel

[illegible]

4.4.3. Textklassifizierungsdatenbank

Schema

Abbildung 10. Textklassifizierungsdatenbank

Inhaltsbeispiel

123 satz_nr	A-Z woerter	A-Z anmerkungen	123 bio	A-Z absichten	A-Z szenarios
169	Sag	-	[NULL]	entfernen	ToDo
169	bitte	-	[NULL]	entfernen	ToDo
169	den	-	[NULL]	entfernen	ToDo
169	Besuch	Aktivität	1	entfernen	ToDo
169	bei	Aktivität	1	entfernen	ToDo
169	meinen	Aktivität	1	entfernen	ToDo
169	Großeltern	Aktivität	1	entfernen	ToDo
169	für	-	[NULL]	entfernen	ToDo
169	Sonntag	Datum	1	entfernen	ToDo
169	ab	-	[NULL]	entfernen	ToDo
170	Wie	-	[NULL]	abfragen	Wetter
170	ist	-	[NULL]	abfragen	Wetter
170	das	-	[NULL]	abfragen	Wetter
170	Wetter	-	[NULL]	abfragen	Wetter
170	morgen	Datum	1	abfragen	Wetter
170	früh	Zeit	1	abfragen	Wetter

Abbildung 11. Textklassifizierungsdatenbankinhalt

5. Testing und Evaluation

5.1. Teststrategie und -methoden

Um die Leistungsfähigkeit des lokalen KI-Sprachassistenten zu bewerten, wurde eine umfassende Teststrategie entwickelt. Diese umfasst:

- **Funktionale Tests:** Überprüfung der korrekten Ausführung von Befehlen und Anfragen.
- **Benutzertests:** Bewertung der Benutzerfreundlichkeit und Genauigkeit der Spracherkennung (Aktivierungswortmodells und Authentifizierungsmodells).
 - Die Tests wurden mit einem diversen Datensatz durchgeführt, der verschiedene Sprachgeschwindigkeiten und Umgebungsgeräusche abdeckt.

5.2. Ergebnisse der Funktions- und Leistungstests

5.2.1. Ausgabeergebnisse

Die Ergebnisse der Tests zeigen, dass der Sprachassistent in der Lage ist, die meisten Anfragen korrekt zu verarbeiten und angemessene Antworten zu liefern. Die wichtigsten Erkenntnisse sind:

Notenverteilung

Die Assistenten wurden mit 195 verschiedenen Fragen getestet.

Szenario	Testdatenmenge
Wetter	8
Studienordnung	57
Wikipedia	55
Todo	56
Datum	11
Youtube	3
Search Engine	5
Gesamttestmenge	195

Nachfolgend sind die erzielten Ergebnisse dargestellt:

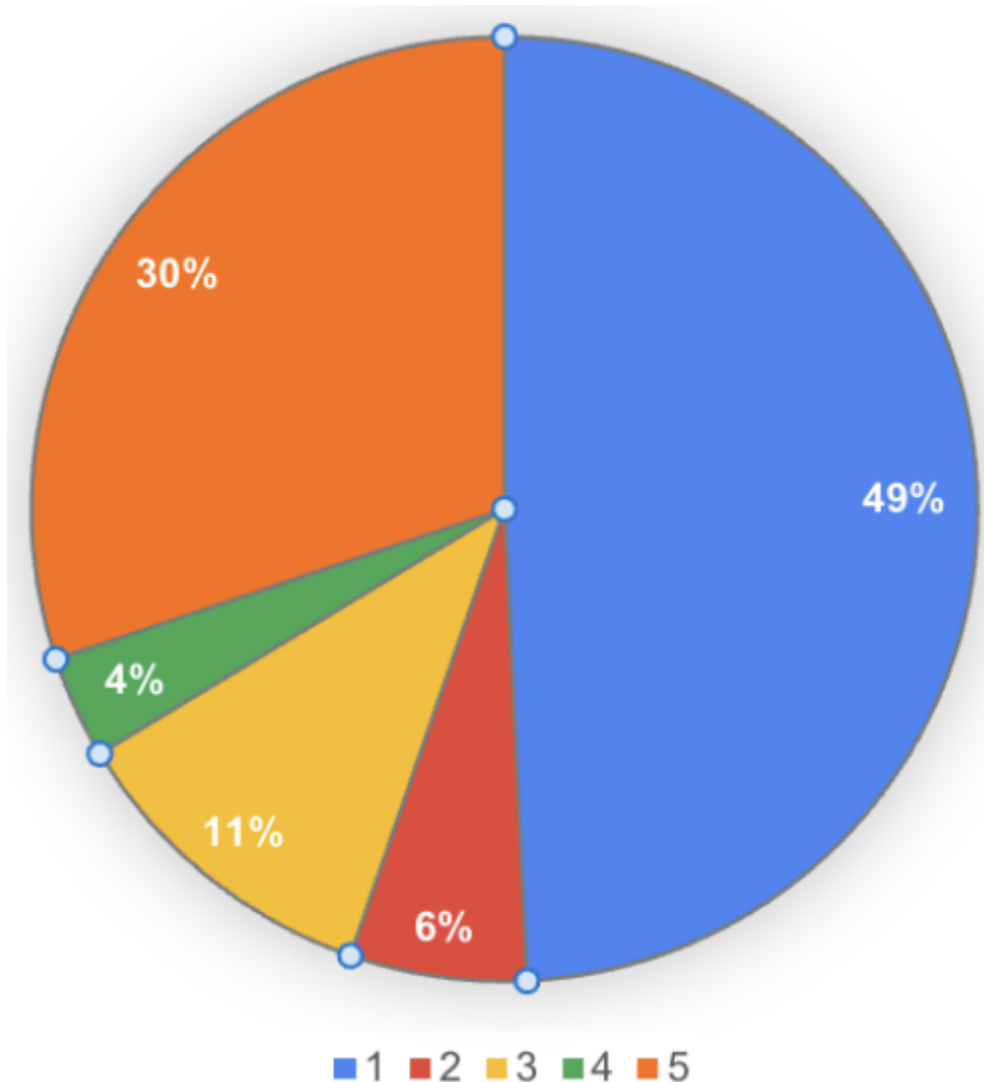


Abbildung 12. Notenverteilung

Beispielausgaben

Szenario	Eingabe	Ausgabe	Note
Wetter	Wie ist das Wetter am 11.12. um 12 Uhr?	in Dresden ist es am 11.12. um 12:00 Uhr Bedeckt bei 1 Grad	1
Studienordnung	Wie lange dauert das Studium eigentlich?	Die Regelstudienzeit für das Vollzeitstudium beträgt 6 Semester.	1
Wikipedia	In welchem Land wurde Marie Curie geboren?	Marie Curie wurde in Warschau, Russisches Kaiserreich (heute Polen) geboren.	1
Wikipedia	Was bedeutet Python?	"Universal nutzbare höhere Programmiersprache".	3

Szenario	Eingabe	Ausgabe	Note
Todo	Ich habe ein Meeting mit Tim am übernächsten Freitag	Ich bin für diese Absicht noch nicht trainiert!	5
Studienordnung	Welche Module haben im ersten Semester nur Vorlesungen?	Da die genauen Details nicht explizit angegeben sind, ist es schwer, eine präzise Antwort zu geben.	5

- **Fehleranalyse:** Die häufigsten Fehler treten bei den Intents für Wikipedia und die Studienordnung auf.

5.2.2. Konfusionsmatrix

Zur Bewertung der Klassifikationsgenauigkeit des Sprachassistenten wurde eine Konfusionsmatrix erstellt. Diese zeigt, wie oft die erkannten Sprachbefehle korrekt oder falsch klassifiziert wurden.

- Diagonale Werte: Zeigen die korrekt klassifizierten Befehle an.
- Nicht-diagonale Werte: Zeigen Fehlklassifikationen an.

Konfusionsmatrix der Textklassifizierungsmodells

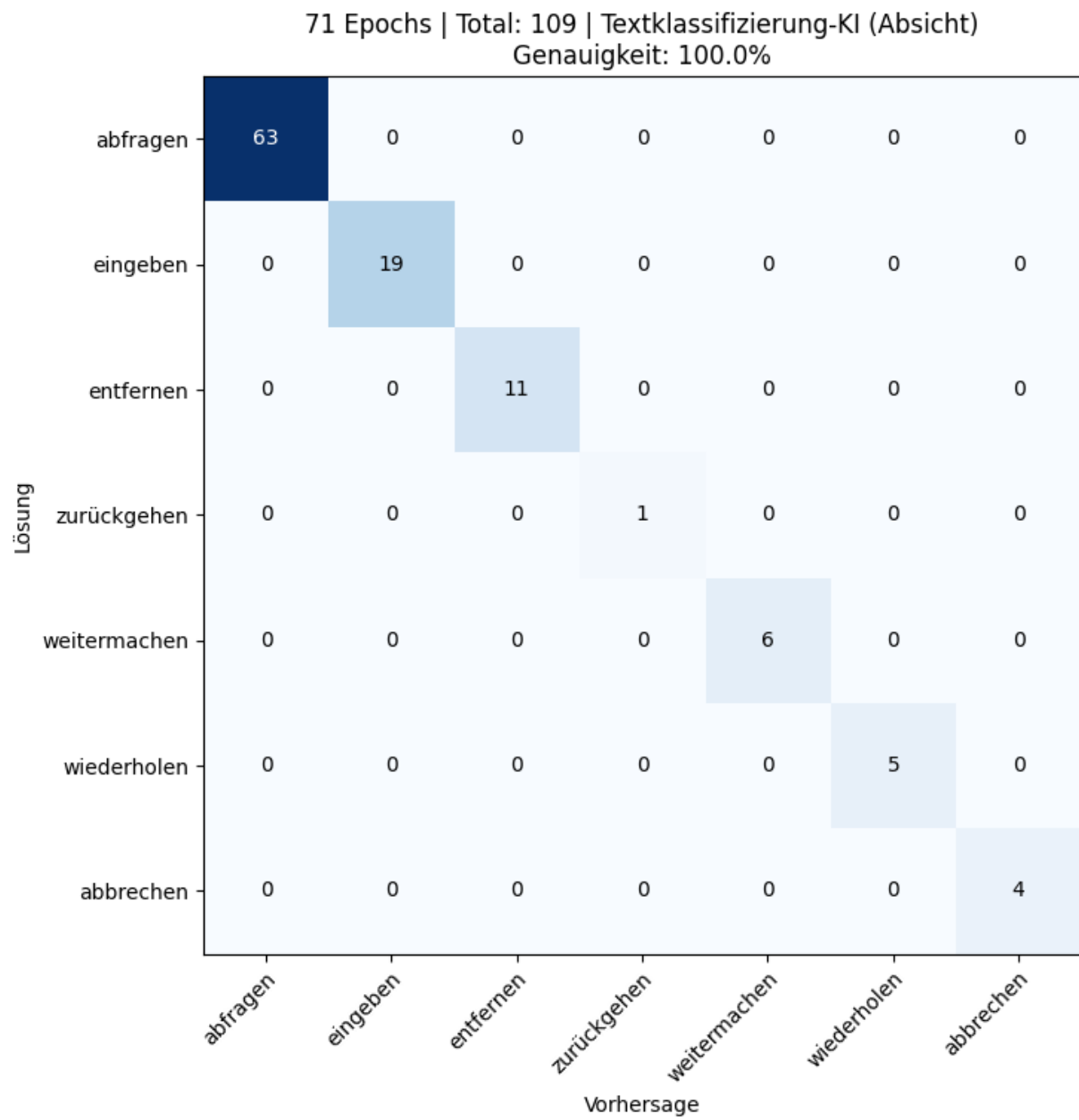


Abbildung 13. Absicht

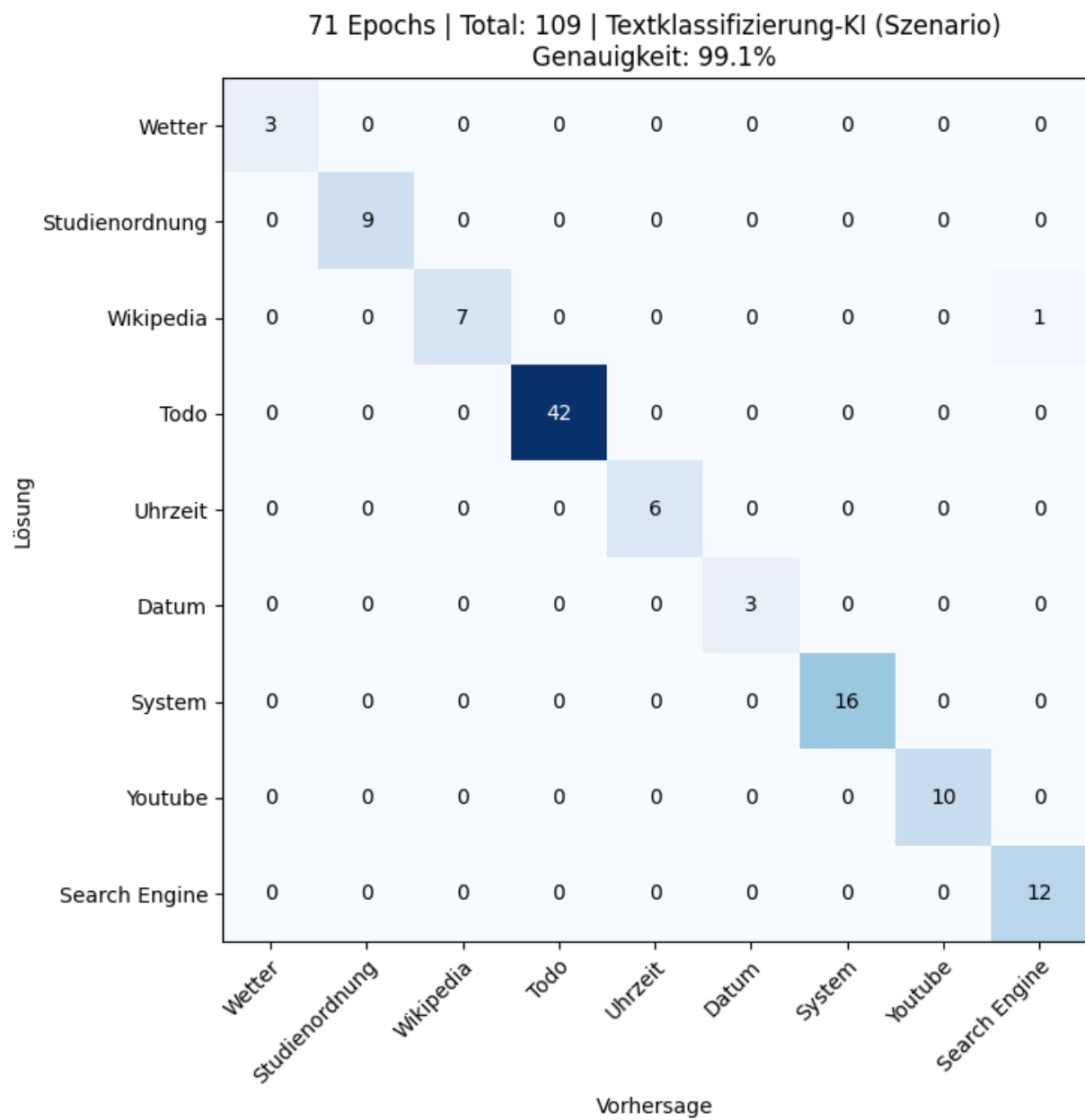


Abbildung 14. Szenario

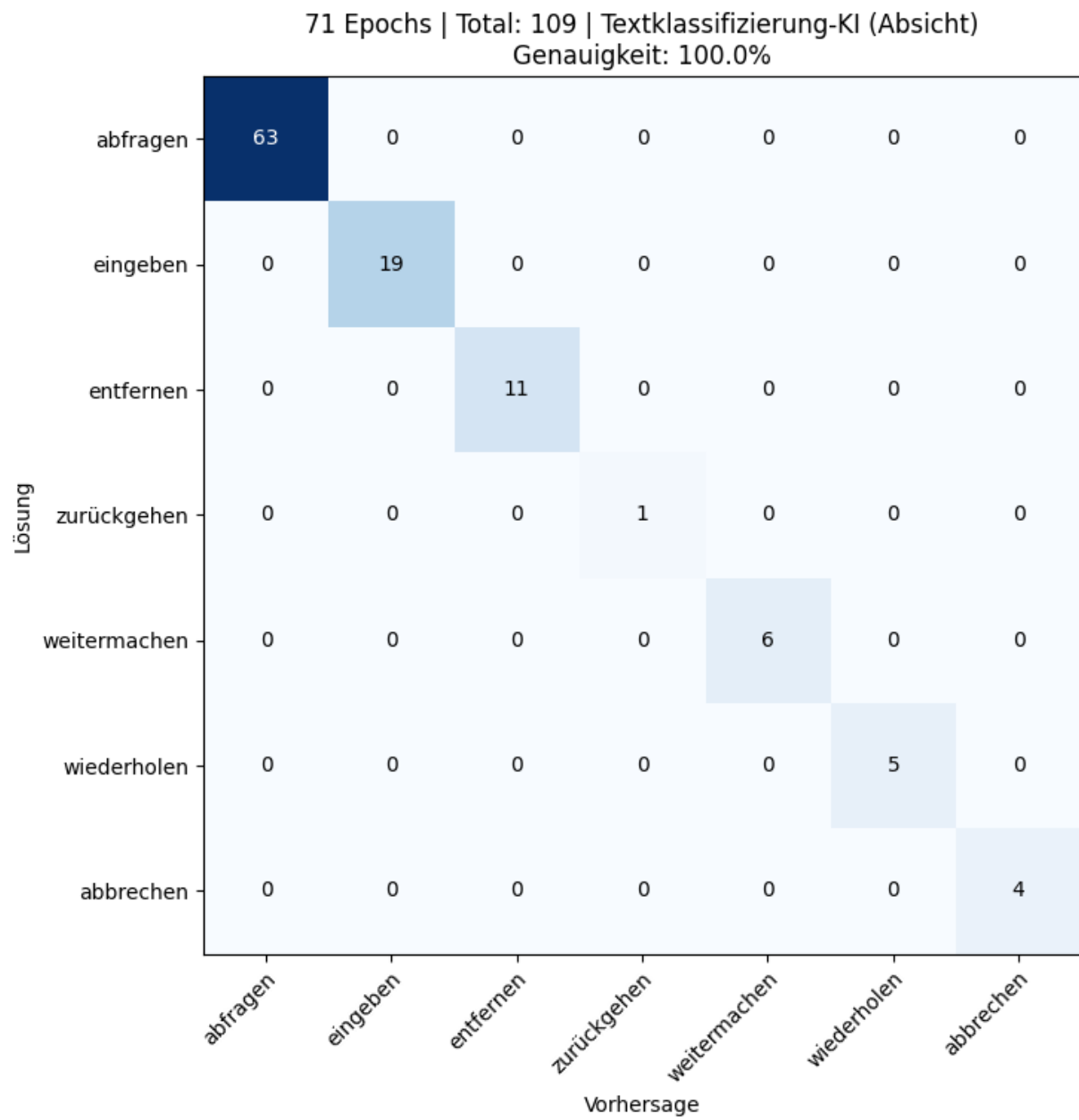


Abbildung 15. Anmerkung

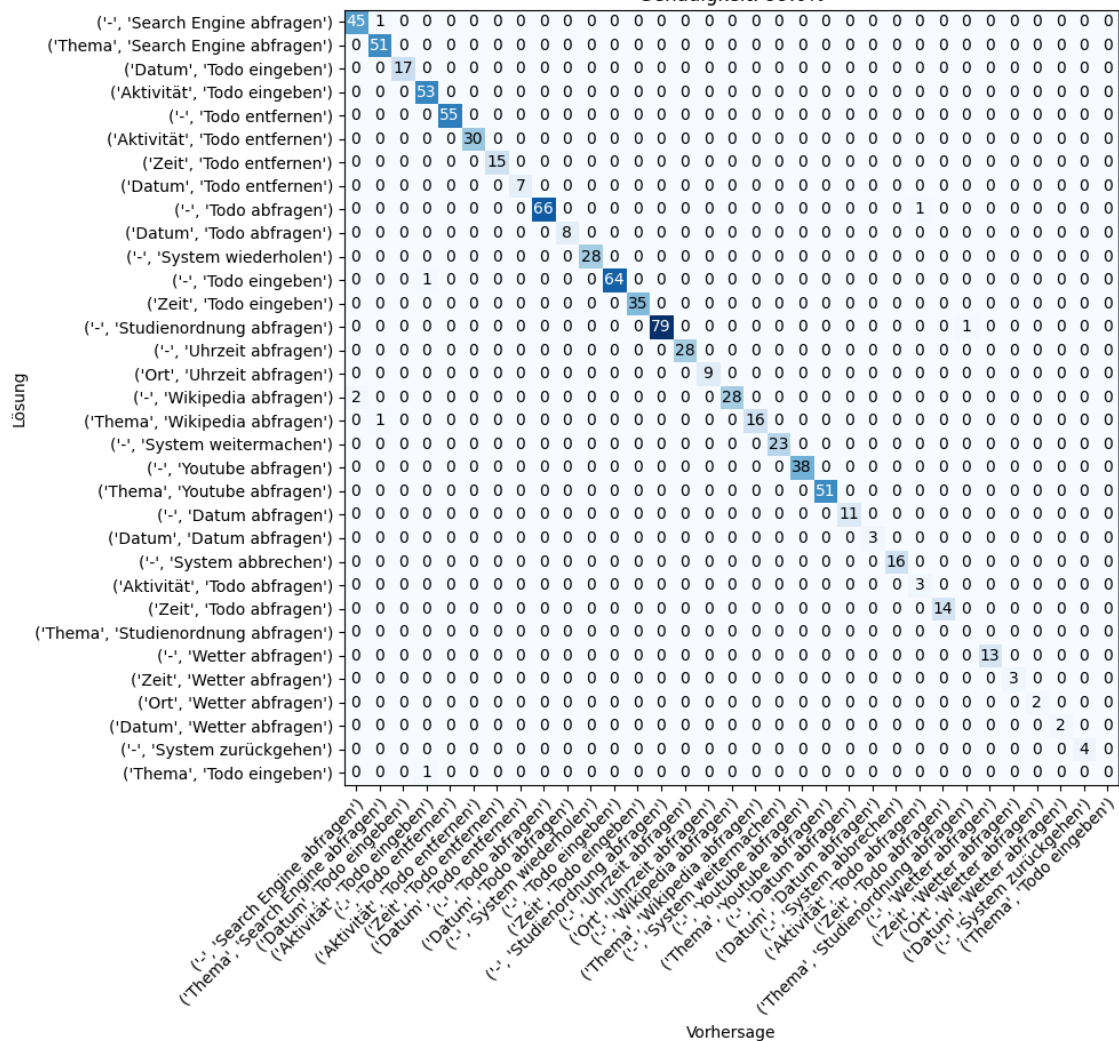


Abbildung 16. Anmerkung & Absicht & Szenario

Die Konfusionsmatrix für das Textklassifizierungsmodell zeigt eine beeindruckende Genauigkeit von über 99%, was darauf hindeutet, dass das Modell in der überwiegenden Mehrheit der Fälle die richtigen Absichten und Szenarien korrekt identifizieren konnte. Dies ist ein hervorragendes Ergebnis und unterstreicht die Effektivität des Modells bei der Verarbeitung von Benutzeranfragen. Allerdings gibt es einige wenige Fehlklassifikationen, die auf bestimmte Herausforderungen zurückzuführen sind.

Mögliche Ursachen für Fehlklassifikationen

- **Ähnlich klingende Befehle:** Einige Befehle sind semantisch oder syntaktisch sehr ähnlich, was die Unterscheidung für das Modell erschwert. Zum Beispiel könnten Fragen wie "Wie lange dauert das Studium?" und "Wie viele Semester hat das Studium?" vom Modell leicht verwechselt werden, da sie sich auf sehr ähnliche Themen beziehen.
- **Komplexe Anfragen:** Komplexe oder mehrdeutige Anfragen, die eine hohe Kontextabhängigkeit erfordern, können zu Fehlklassifikationen führen. Zum Beispiel könnte eine Frage wie "Was muss ich im ersten Semester beachten?" je nach Kontext unterschiedliche Antworten erfordern, was das Modell verwirren könnte.
- **Fehler im Datensatz:** Da der Datensatz selbst erstellt wurde, könnten möglicherweise Fehler oder Ungenauigkeiten im Datensatz vorhanden sein. Dies könnte beispielsweise falsch

annotierte Daten oder unzureichende Variationen in den Trainingsdaten umfassen. Solche Fehler im Datensatz können die Leistung des Modells beeinträchtigen, selbst wenn das Modell an sich gut trainiert ist.

- **Begrenzte Trainingsdaten:** Obwohl der Datensatz umfangreich ist, könnte es dennoch Lücken geben, insbesondere bei seltenen oder spezifischen Anfragen. Ein begrenzter oder unausgewogener Datensatz kann dazu führen, dass das Modell in bestimmten Szenarien weniger genau arbeitet.

Konfusionsmatrix der Aktivierungswortmodells

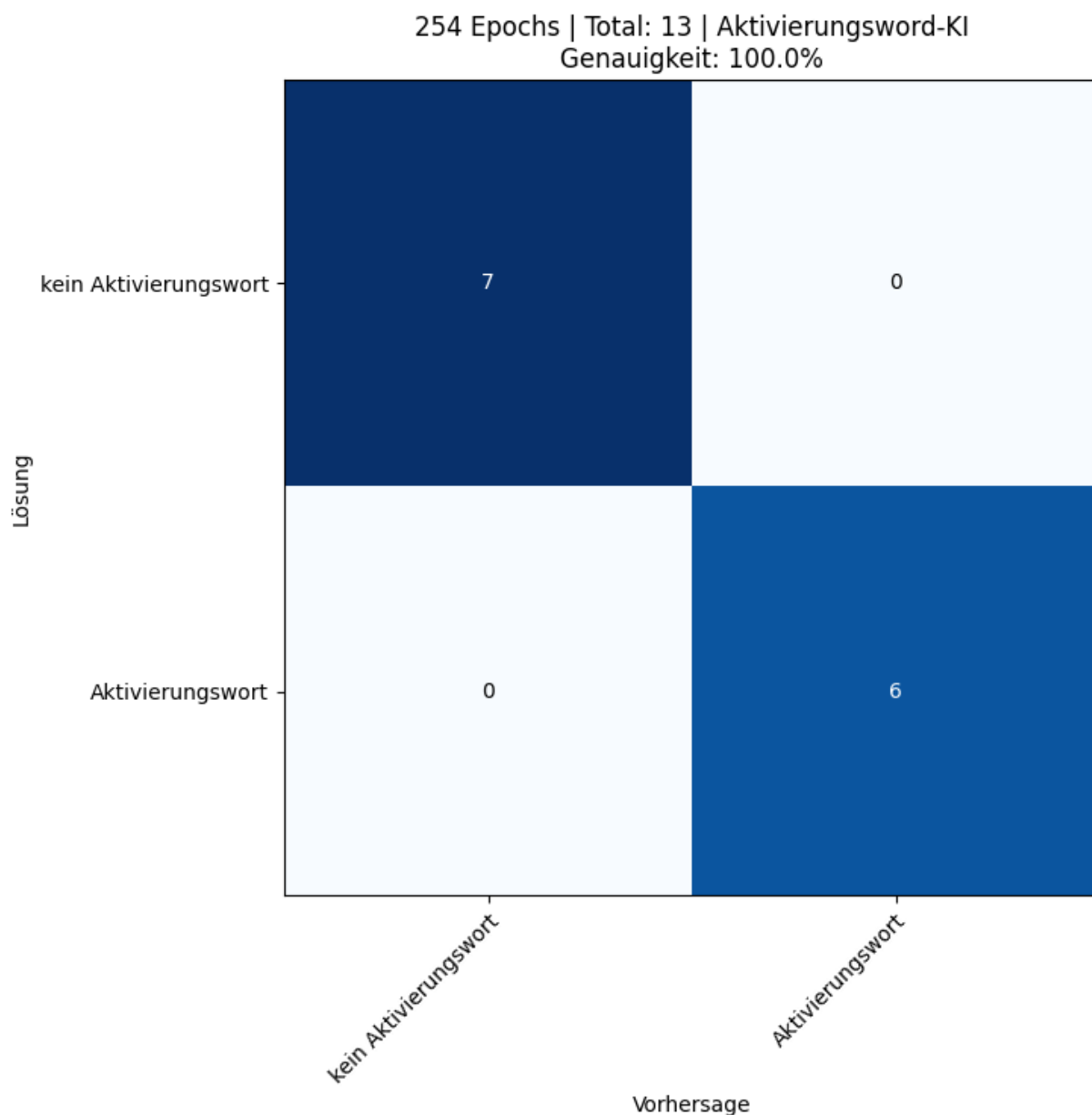


Abbildung 17. Aktivierungswort

Das Aktivierungswortmodell, das für die Erkennung des Aktivierungsworts (z.B. "Hey Assistent") verantwortlich ist, hat in den durchgeführten Tests eine perfekte Genauigkeit von 100% erreicht. Dies bedeutet, dass das Modell in allen Testfällen das Aktivierungswort korrekt erkannt hat, ohne falsch positive oder falsch negative Ergebnisse. Dies ist ein herausragendes Ergebnis und

unterstreicht die Robustheit und Zuverlässigkeit des Modells.

Konfusionsmatrix der Authentifizierungsmodells

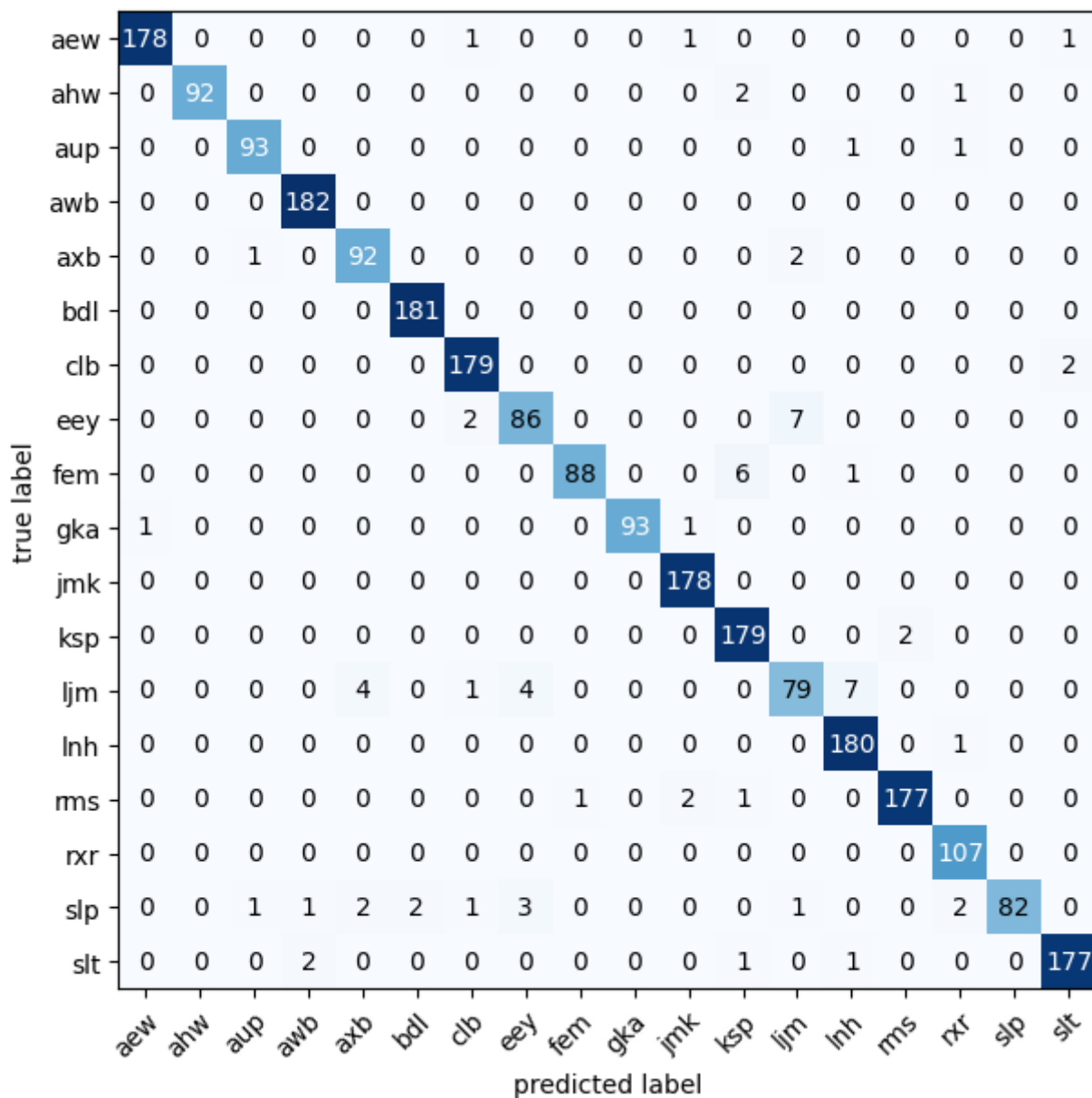


Abbildung 18. Authentifizierung

Das Authentifizierungsmodell, das für die Benutzerauthentifizierung verantwortlich ist, zeigte ebenfalls eine sehr hohe Genauigkeit. Dies bedeutet, dass das Modell in der Lage war, die Stimme des Benutzers korrekt zu identifizieren und unbefugte Zugriffe effektiv zu verhindern.

6. Zukunftsperspektiven

Das Projekt des lokalen KI-Sprachassistenten hat bereits vielversprechende Ergebnisse geliefert, jedoch gibt es zahlreiche Möglichkeiten zur Weiterentwicklung und Verbesserung. Im Folgenden werden die zukünftigen Perspektiven und Erweiterungsmöglichkeiten beschrieben:

6.1. Unterstützung für mehrere Benutzer

Aktuell ist der Sprachassistent nur für einen einzelnen Benutzer ausgelegt. Die gleichzeitige Nutzung durch mehrere Benutzer kann zu Konflikten führen, insbesondere bei der Spracherkennung und der Verarbeitung von Anfragen. Zukünftig könnte das System um eine Benutzerverwaltung erweitert werden, die es ermöglicht, mehrere Nutzer zu unterscheiden und personalisierte Antworten bereitzustellen. Dies könnte durch die Integration von Benutzerprofilen und einer robusten Konfliktlösungsstrategie erreicht werden.

6.2. Verbesserte Spracherkennung

Derzeit basiert die Spracherkennung (Speech-to-Text) und Sprachsynthese (Text-to-Speech) auf Open-Source-Lösungen wie Coqui TTS und Whisper. Obwohl diese Technologien leistungsfähig sind, gibt es noch Verbesserungspotenzial in Bezug auf Robustheit, Genauigkeit und Latenz. Zukünftig könnten fortschrittlichere Modelle oder ein Fine-Tuning der bestehenden Systeme eingesetzt werden, um die Sprachverarbeitung in lauten Umgebungen oder bei Akzenten zu optimieren.

6.3. Bessere Informationsanalyse

Um die Qualität der Antworten zu verbessern, könnte ein Fine-Tuning von Large Language Models (LLMs) für spezifische Intents wie "WikipediaIntent" oder "StudienordnungIntent" durchgeführt werden. Dies würde es dem Assistenten ermöglichen, präzisere und kontextbezogene Antworten zu liefern, insbesondere bei komplexen Anfragen. Durch das Training auf spezialisierten Datensätzen könnte der Assistent besser auf akademische oder informatorische Anfragen reagieren.

6.4. Implementierung von LLMs

Die Integration von modernen LLMs (z. B. GPT, LLaMA oder andere Open-Source-LLMs) könnte die Qualität der generierten Antworten erheblich steigern. Diese Modelle könnten nicht nur für die Beantwortung von Fragen, sondern auch für die Generierung von natürlich klingenden Dialogen und die Bereitstellung von detaillierten Erklärungen genutzt werden. Dies würde die Benutzererfahrung deutlich verbessern.

6.5. Erweiterung der Intents

Derzeit unterstützt der Sprachassistent eine begrenzte Anzahl von Intents. Zukünftig könnten weitere Intents hinzugefügt werden, um den Funktionsumfang zu erweitern. Beispielsweise könnten Intents für die Steuerung von IoT-Geräten (z. B. Smart Home-Geräte) implementiert

werden. Dies würde den Assistenten zu einem zentralen Steuerungspunkt für vernetzte Geräte machen und seine Nützlichkeit im Alltag erhöhen.

7. Fazit

Das Projekt hat gezeigt, dass ein effizienter und datenschutzfreundlicher KI-Sprachassistent vollständig auf lokaler Hardware betrieben werden kann. Durch den Einsatz moderner NLP- und ML-Technologien wurde ein System entwickelt, das Sprachbefehle zuverlässig versteht und ausführt – ohne externe Datenübertragung.

Herausforderungen wie begrenzte Rechenleistung, Echtzeitverarbeitung der Spracherkennung und die Integration verschiedener Softwarekomponenten wurden erfolgreich gelöst. Dabei erwies sich eine durchdachte Systemarchitektur als essenziell, um die Komplexität zu bewältigen. Zudem zeigte sich, dass die Qualität der Spracherkennung maßgeblich von der Hardware und den trainierten Modellen abhängt.

In Zukunft könnten Optimierungen bei Modellgröße und Effizienz sowie eine breitere Datenbasis die Leistung weiter steigern.

8. Anhang

8.1. Glossar

- **KI (Künstliche Intelligenz):** Technologie, die es Maschinen ermöglicht, menschenähnliche Intelligenz zu zeigen.
- **NLP (Natural Language Processing):** Teilgebiet der KI, das sich mit der Interaktion zwischen Computern und menschlicher Sprache befasst.
- **Lokale Verarbeitung:** Datenverarbeitung auf dem Gerät des Benutzers, ohne Cloud-Dienste zu nutzen.
- **API (Application Programming Interface):** Schnittstelle zur Integration von Softwarekomponenten.
- **Open Source:** Software, deren Quellcode öffentlich zugänglich ist und frei modifiziert werden kann.
- **Absicht:** Die vom Benutzer gewünschte Aktion, die aus einer Sprach- oder Texteingabe extrahiert wird. Beispiele für Absichten sind Verben wie löschen, abfragen, hinzufügen oder bearbeiten.
- **Szenario:** Der Kontext oder das Thema, in dem die Absicht ausgedrückt wird. Beispiele für Szenarien sind Todo-Liste, Wetter, Wikipedia, Kalender oder Musikkwiedergabe.
- **Anmerkung/Entität:** Ein spezifisches Datenelement, das aus einer Benutzereingabe extrahiert wird (z. B. Datum, Ort, Name).
- **Intent:** Die Kombination aus Szenario und Absicht, die die vollständige Bedeutung einer Benutzereingabe beschreibt. Beispiel: "Füge einen Termin hinzu" (Szenario: Kalender, Absicht: hinzufügen).

9. Literaturverzeichnis

9.1. Allgemeine Quellen zu KI

- Jurafsky, D., & Martin, J. H. (3rd ed.). *Speech and Language Processing*. Stanford University. Verfügbar unter: <https://web.stanford.edu/~jurafsky/slp3/> [Zugriff: November 2024]
- Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. MIT Press. Verfügbar unter: <https://www.deeplearningbook.org/> [Zugriff: November 2024]
- Chollet, F. *Deep Learning with Python* (2nd ed.). Manning Publications. Verfügbar unter: <https://www.manning.com/books/deep-learning-with-python-second-edition> [Zugriff: November 2024]
- Freiknecht, Jonas. *KI-Sprachassistenten mit Python entwickeln: Datenbewusst, Open Source und modular*. Hanser.

9.2. Audioverarbeitung und Feature-Extraktion

- MasazI. *Audio Classification and Feature Extraction*. Verfügbar unter: https://github.com/MasazI/audio_classification/blob/master/features.py [Zugriff: November 2024]
- Librosa. *Audio and Music Analysis in Python*. Verfügbar unter: <https://librosa.org/> [Zugriff: November 2024]

9.3. Microservices und Threading

- Richardson, C. *Microservices Patterns: With Examples in Java*. Manning Publications. Verfügbar unter: <https://www.manning.com/books/microservices-patterns> [Zugriff: November 2024]
- Python Threading Documentation. *concurrent.futures — Asynchronous Execution*. Verfügbar unter: <https://docs.python.org/3/library/concurrent.futures.html> [Zugriff: November 2024]

9.4. Open-Source-Projekte

- Thorsten-Voice. *Open-Source Voice Assistant*. Verfügbar unter: <https://github.com/thorstenMueller/Thorsten-Voice> [Zugriff: November 2024]
- OpenAI Whisper. *Robust Speech Recognition via Large-Scale Weak Supervision*. Verfügbar unter: <https://github.com/openai/whisper> [Zugriff: November 2024]
- Coqui TTS. *Text-to-Speech Library*. Verfügbar unter: <https://github.com/coqui-ai/TTS> [Zugriff: November 2024]
- Hugging Face. *BERT. Transformers Documentation*. Hugging Face. Verfügbar unter: https://huggingface.co/docs/transformers/model_doc/bert [Zugriff: November 2024]

9.5. Frameworks und Tools

- PyTorch. *Deep Learning Framework*. Verfügbar unter: <https://pytorch.org/docs/stable/index.html>

[Zugriff: November 2024]

- CNN Explainer. *Visualization of Convolutional Neural Networks*. Verfügbar unter: <https://poloclub.github.io/cnn-explainer/> [Zugriff: November 2024]

9.6. Datenschutz und Sicherheit

- GDPR. *General Data Protection Regulation*. Verfügbar unter: <https://gdpr-info.eu/> [Zugriff: November 2024]
- NIST. *Cybersecurity Framework*. Verfügbar unter: <https://www.nist.gov/cyberframework> [Zugriff: November 2024]

9.7. Hochschulbezogene Quellen

- HTW Dresden. *Ordnungen und Satzungen*. Verfügbar unter: <https://www.htw-dresden.de/studium/im-studium/ordnungen-und-satzungen> [Zugriff: November 2024]

9.8. Sonstige Quellen

- YouTube. *Verschiedene Tutorials und Erklärvideos*. Verfügbar unter: <https://www.youtube.com> [Zugriff: November 2024]
- Stack Overflow. *Community-gestützte Fragen und Antworten zur Programmierung*. Verfügbar unter: <https://stackoverflow.com> [Zugriff: November 2024]
- OpenAI. *ChatGPT*. Verfügbar unter: <https://chat.openai.com> [Zugriff: November 2024]
- Medium. *Artikel und Blogbeiträge zu KI, Programmierung und Technologie*. Verfügbar unter: <https://medium.com> [Zugriff: November 2024]